Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп’ютерної інженерії
Кафедра ПЗ

Практична робота №3

з дисципліни «Розробка проєктів засобами платформи .NET»

Виконав: ст. 1ПІ-24Б Кузя Н.І.
Перевірив: викл. Позур М.Ю.

м. Вінниця – 2026

Тема: Узагальнення, делегати, події та лямбда-вирази

Варіант: 3 (Відстеження посилок)

Хід роботи

1. Короткий опис реалізованого функціоналу

У даній роботі розроблено консольний застосунок для відстеження статусу посилок. Було реалізовано інтерфейс IShipment та клас Parcel. Створено узагальнений клас Tracker<T> з обмеженням типу (where T : class, IShipment), який інкапсулює логіку збереження, обробки та зміни статусу відправлень. Для обчислення вартості доставки за різними тарифами використано делегат Func<T, decimal> (необов'язкове правило). Сповіщення про зміну статусу посилки реалізовано за допомогою події StatusChanged за стандартним шаблоном .NET (з використанням класу StatusChangedEventArgs). Програма містить інтерактивне меню для додавання посилок, зміни їх статусів та вибору тарифів (стандартний, експрес, фіксований, акційний).

2. Фрагмент коду: "іменований метод vs лямбда-вираз"

```
// 1. Іменований метод
tracker.StatusChanged += LogToHistory;

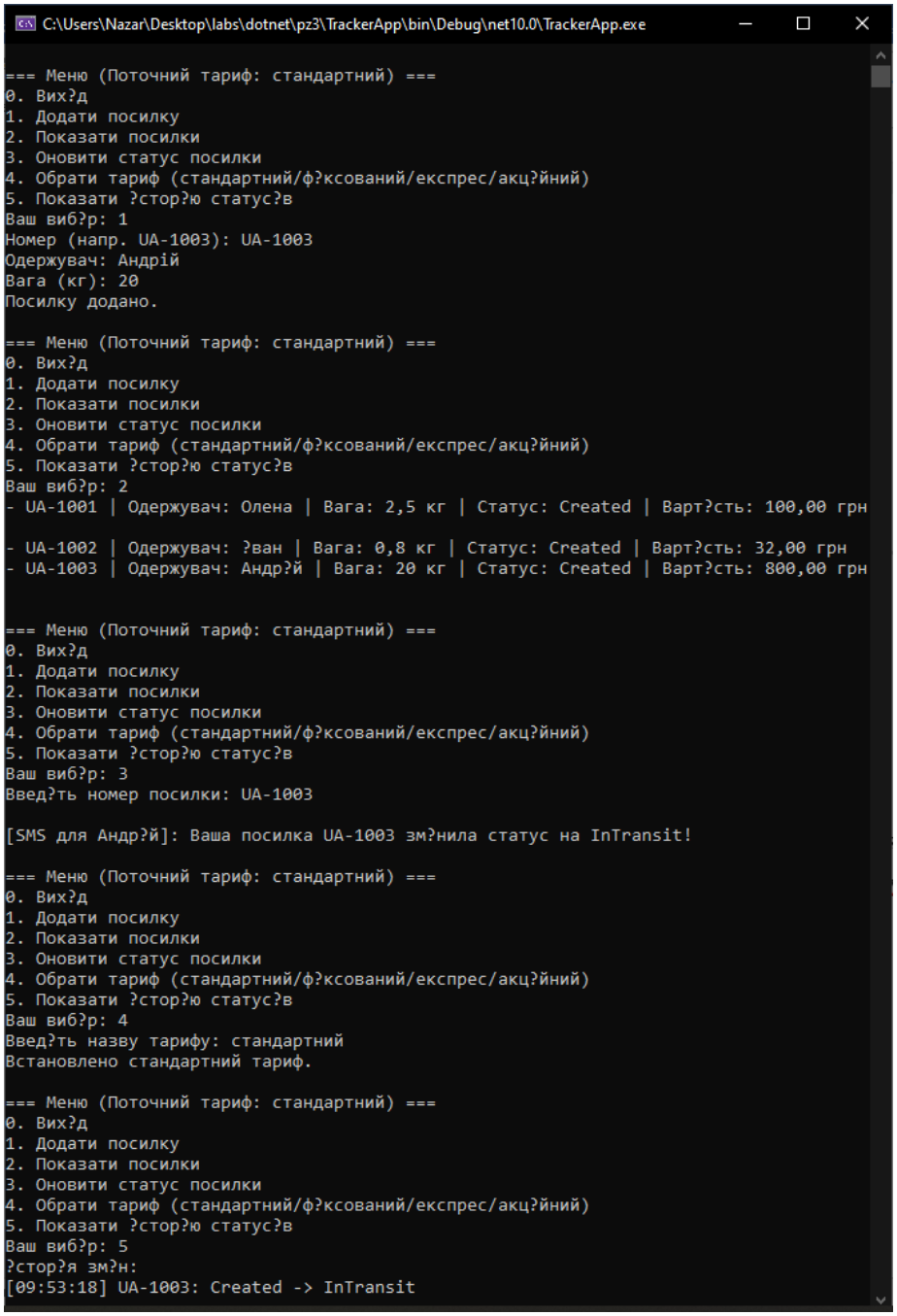
static void LogToHistory(object? sender, StatusChangedEventArgs<Parcel> e)
{
   history.Add($"[{DateTime.Now:HH:mm:ss}] {e.Item.TrackingNumber}:
{e.OldStatus} -> {e.NewStatus}");
}

// 2. Лямбда-вираз
tracker.StatusChanged += (sender, e) =>
{
   if (e.Item is Parcel p)
   {
      Console.WriteLine($"[SMS]: Ваша посилка {p.TrackingNumber} змінила
статус на {e.NewStatus}!");
```

```
    }
};
```

Пояснення: Іменований метод (LogToHistory) доречніше використовувати для логіки, яка може бути перевикористана в інших частинах програми, або якщо від події потрібно буде згодом відписатися (оператор -=). Лямбда-вираз ідеально підходить для коротких, одноразових реакцій (як виведення SMS на екран), де немає потреби створювати окремий метод, і логіка читається безпосередньо в місці підписки.

Результат роботи програми:



3. Посилання на репозиторій GitHub

https://github.com/shwballl/vntu-dotnet

4. Лістинг коду (Program.cs)

```
using System;
using System.Collections.Generic;

public enum ShipmentStatus { Created, InTransit, Arrived, Received }

public interface IShipment
{
    string TrackingNumber { get; }
    decimal WeightKg { get; }
    ShipmentStatus Status { get; set; }
}

public class Parcel : IShipment
{
    public string TrackingNumber { get; }
    public decimal WeightKg { get; }
    public ShipmentStatus Status { get; set; }
    public string Recipient { get; }

    public Parcel(string trackingNumber, decimal weightKg, string recipient)
    {
        TrackingNumber = trackingNumber;
        WeightKg = weightKg;
        Recipient = recipient;
        Status = ShipmentStatus.Created;
    }
}

public class StatusChangedEventArgs<T> : EventArgs
```

```
{
    public T Item { get; }
    public ShipmentStatus OldStatus { get; }
    public ShipmentStatus NewStatus { get; }

    public StatusChangedEventArgs(T item, ShipmentStatus oldStatus, ShipmentStatus
newStatus)
    {
        Item = item;
        OldStatus = oldStatus;
        NewStatus = newStatus;
    }
}

public class Tracker<T> where T : class, IShipment
{
    private readonly List<T> _items = new();
    private Func<T, decimal>? _tariff;

    public event EventHandler<StatusChangedEventArgs<T>>? StatusChanged;

    public void Add(T item) => _items.Add(item);

    public bool AdvanceStatus(string trackingNumber)
    {
        var item = _items.Find(x => x.TrackingNumber == trackingNumber);
        if (item == null) return false;
        if (item.Status == ShipmentStatus.Received) return true;

        var oldStatus = item.Status;
        item.Status = (ShipmentStatus)((int)item.Status + 1);
```

```
        OnStatusChanged(new StatusChangedEventArgs<T>(item, oldStatus,
item.Status));
        return true;
    }

    public decimal GetCost(T item) => _tariff?.Invoke(item) ?? (item.WeightKg *
40m);

    public void ForEach(Action<T> action)
    {
        foreach (var item in _items) action(item);
    }

    public void SetTariff(Func<T, decimal>? tariff) => _tariff = tariff;

    protected virtual void OnStatusChanged(StatusChangedEventArgs<T> e) =>
StatusChanged?.Invoke(this, e);
}

class Program
{
    static List<string> history = new();

    static void Main()
    {
        var tracker = new Tracker<Parcel>();
        tracker.Add(new Parcel("UA-1001", 2.5m, "Олена"));
        tracker.Add(new Parcel("UA-1002", 0.8m, "Іван"));

        tracker.StatusChanged += LogToHistory;
        tracker.StatusChanged += (sender, e) =>
        {
```

```
        if (e.Item is Parcel p)
        {
            Console.WriteLine($"\n[SMS для {p.Recipient}]: Ваша посилка
{p.TrackingNumber} змінила статус на {e.NewStatus}!");
        }
    };

    var tariffs = new Dictionary<string, Func<Parcel, decimal>>
    {
        ["фіксований"] = p => 70m,
        ["експрес"] = p => 100m + p.WeightKg * 60m,
        ["акційний"] = p => p.WeightKg * 40m * 0.8m
    };

    bool isRunning = true;
    while (isRunning)
    {
        Console.WriteLine("\n=== Меню ===");
        Console.WriteLine("0. Вихід");
        Console.WriteLine("1. Додати посилку");
        Console.WriteLine("2. Показати посилки");
        Console.WriteLine("3. Оновити статус посилки");
        Console.WriteLine("4. Обрати тариф");

        Console.Write("Ваш вибір: ");
        string? choice = Console.ReadLine();

        switch (choice)
        {
            case "1":
                tracker.Add(new Parcel("UA-NEW", 1.0m, "Тест"));
                break;
```

```
            case "2":
                tracker.ForEach(p => Console.WriteLine($"- {p.TrackingNumber} |
Статус: {p.Status} | Вартість: {tracker.GetCost(p):F2} грн"));
                break;
            case "3":
                tracker.AdvanceStatus("UA-1001");
                break;
            case "4":
                tracker.SetTariff(tariffs["експрес"]);
                break;
            case "0":
                isRunning = false;
                break;
        }
    }
}

static void LogToHistory(object? sender, StatusChangedEventArgs<Parcel> e)
{
    history.Add($"[{DateTime.Now:HH:mm:ss}] {e.Item.TrackingNumber}:
{e.OldStatus} -> {e.NewStatus}");
}
}
```

Висновок: під час виконання практичної роботи було закріплено навички роботи з узагальненими класами та обмеженнями типів (generics) у C#. Опановано використання вбудованих делегатів (Func, Action), лямбда-виразів та замикань для гнучкого налаштування поведінки об'єктів під час виконання програми (наприклад, динамічна зміна тарифів на доставку). Також було успішно реалізовано механізм подій (events) за стандартним шаблоном .NET для сповіщення підписників про зміну статусу посилки із передачею кастомних аргументів.